

Documentation des Microservices – Architecture De Project Match

Ce document explique chaque service, ses classes principales et les relations entre les classes.

1. Présentation générale

L'architecture est basée sur une approche Microservices. Chaque service possède sa propre base de données (pattern Database per Service) et communique avec les autres services via API REST ou via des événements.

Les services principaux sont :

- **Auth Service** : authentification et gestion des tokens (JWT).
- **Profile Service** : gestion du profil public des utilisateurs.
- **Project Service** : gestion des projets, équipes et demandes de participation.
- **Messaging Service** : messagerie et conversations.
- **Notification Service** : notifications internes et email.
- **Training Service** : gestion des formations, inscriptions et certificats.
- **Admin Service** : modération, signalement et audit.

2. Auth Service

Le **Auth Service** gère l'authentification et l'autorisation des utilisateurs.

Il stocke uniquement les informations sensibles liées au compte (email, mot de passe hashé, rôle, statut du compte).

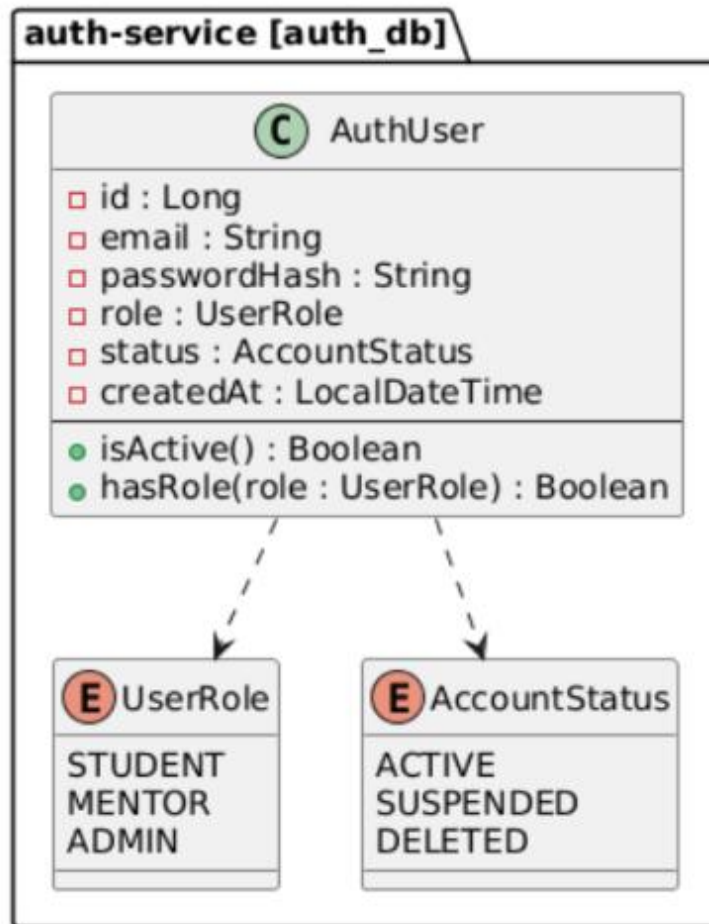
Classes principales

- **AuthUser** : représente un compte utilisateur (identité et sécurité).
- **UserRole (enum)** : définit le rôle de l'utilisateur (STUDENT, MENTOR, ADMIN).
- **AccountStatus (enum)** : définit l'état du compte (ACTIVE, SUSPENDED, DELETED).

Relations entre classes

- **AuthUser** utilise **UserRole** pour définir le rôle de l'utilisateur.
- **AuthUser** utilise **AccountStatus** pour définir l'état du compte.

Auth Service - Diagramme de classes (JWT Stateless)



3. Profile Service (User Service)

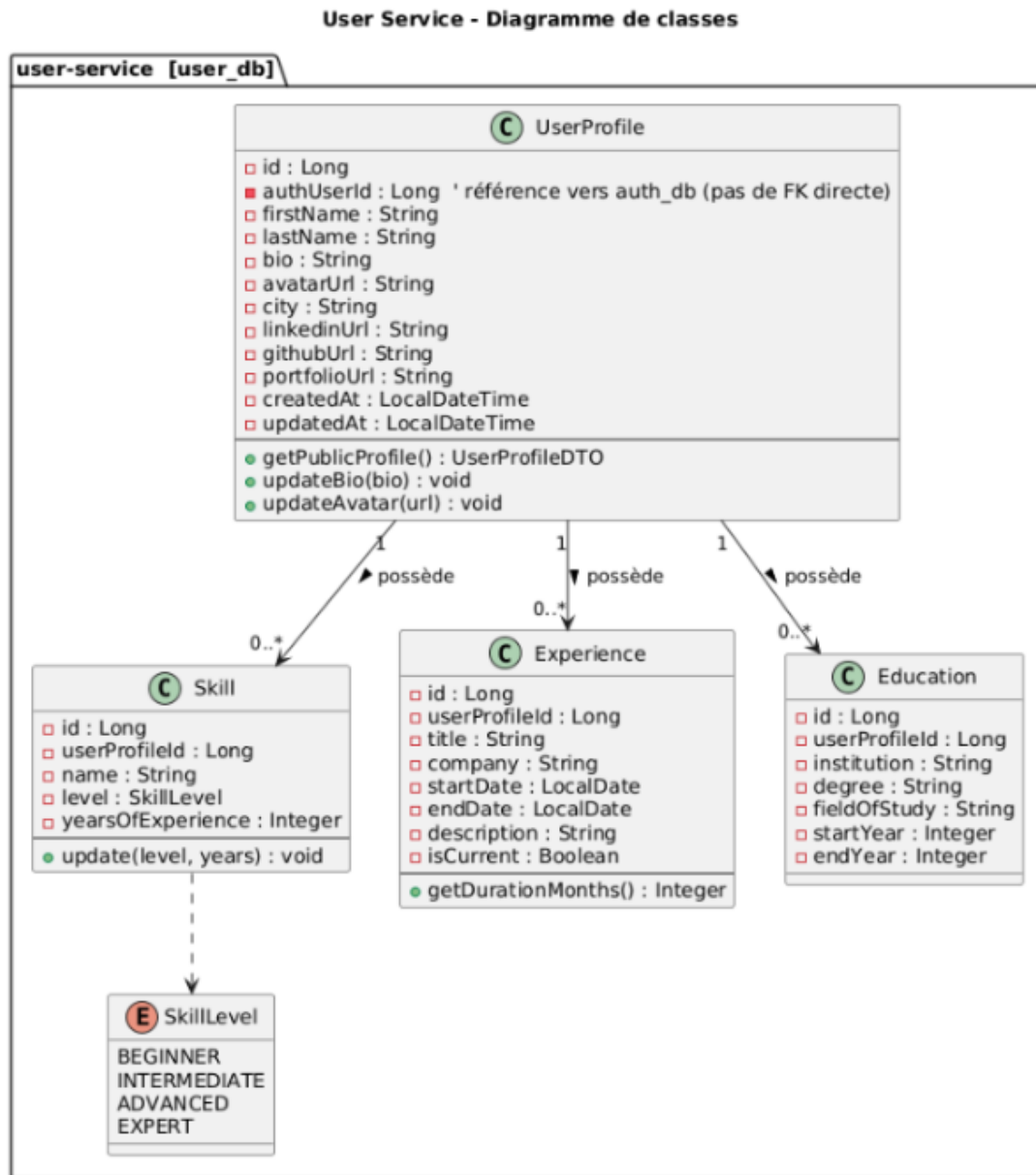
Le Profile Service gère les informations publiques d'un utilisateur. Les données sensibles (email/password) ne sont pas stockées ici. La liaison avec Auth Service se fait via le champ `authUserId`.

Classes principales

- **UserProfile** : informations du profil public (nom, bio, avatar, réseaux sociaux).
- **Skill** : compétence associée au profil.
- **Experience** : expérience professionnelle associée au profil.
- **Education** : formation académique associée au profil.
- **SkillLevel** (enum) : niveau de compétence (`BEGINNER`, `INTERMEDIATE`, `ADVANCED`, `EXPERT`).

Relations entre classes

- UserProfile (1) ---- (0..*) Skill : un profil peut avoir plusieurs compétences.
- UserProfile (1) ---- (0..*) Experience : un profil peut avoir plusieurs expériences.
- UserProfile (1) ---- (0..*) Education : un profil peut avoir plusieurs formations.
- Skill contient userProfileId comme clé étrangère vers UserProfile.



4. Project Service

Le Project Service gère les projets publiés par les utilisateurs, les membres d'équipe, les demandes de participation et les tags. Chaque projet est lié à un propriétaire (ownerId).

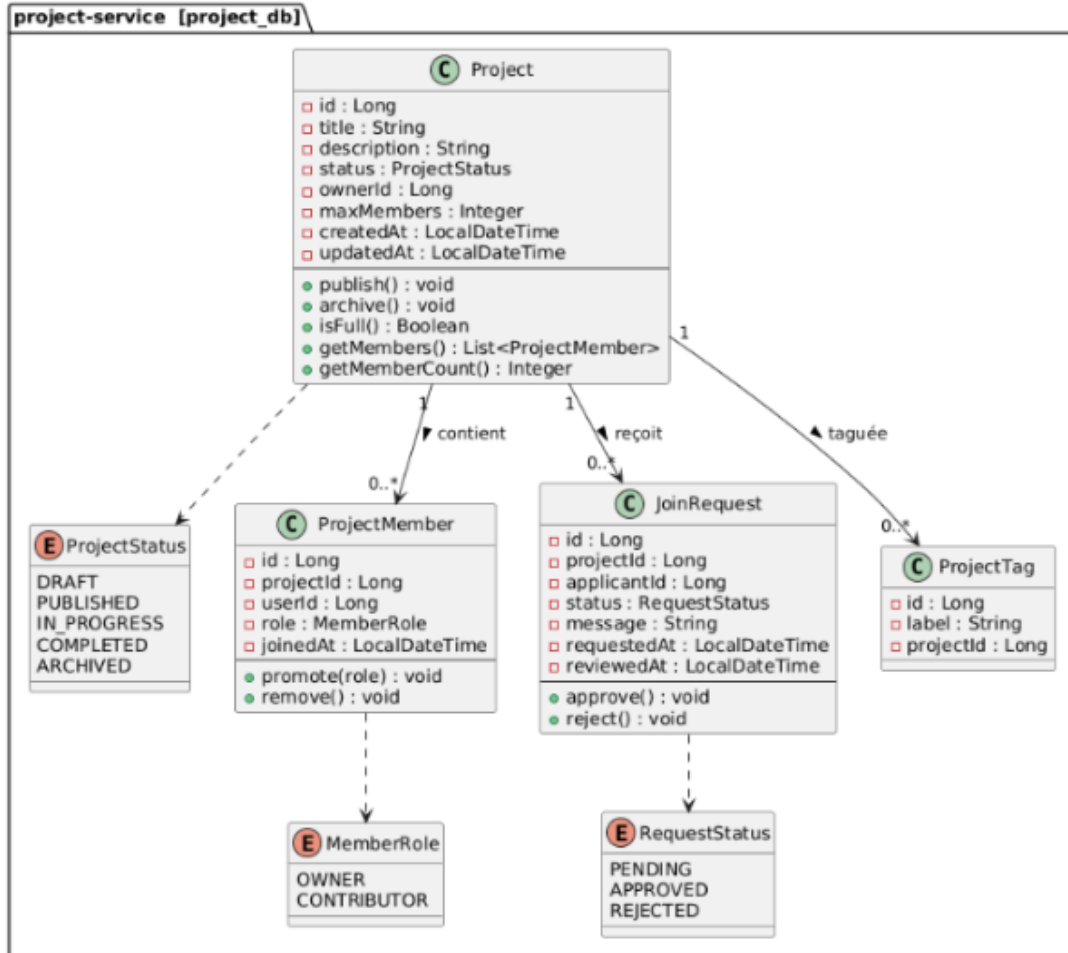
Classes principales

- Project : représente un projet (titre, description, statut, propriétaire).
- ProjectMember : représente un membre d'un projet et son rôle.
- JoinRequest : demande d'adhésion à un projet.
- ProjectTag : tag associé à un projet.
- ProjectStatus (enum) : état du projet (DRAFT, PUBLISHED, IN_PROGRESS, COMPLETED, ARCHIVED).
- RequestStatus (enum) : état de la demande (PENDING, APPROVED, REJECTED).
- MemberRole (enum) : rôle du membre (OWNER, CONTRIBUTOR).

Relations entre classes

- Project (1) ---- (0..*) ProjectMember : un projet contient plusieurs membres.
- Project (1) ---- (0..*) JoinRequest : un projet reçoit plusieurs demandes.
- Project (1) ---- (0..*) ProjectTag : un projet peut avoir plusieurs tags.
- ProjectMember référence Project via projectId.
- JoinRequest référence Project via projectId.
- ProjectTag référence Project via projectId.

Project Service - Diagramme de classes



5. Messaging Service

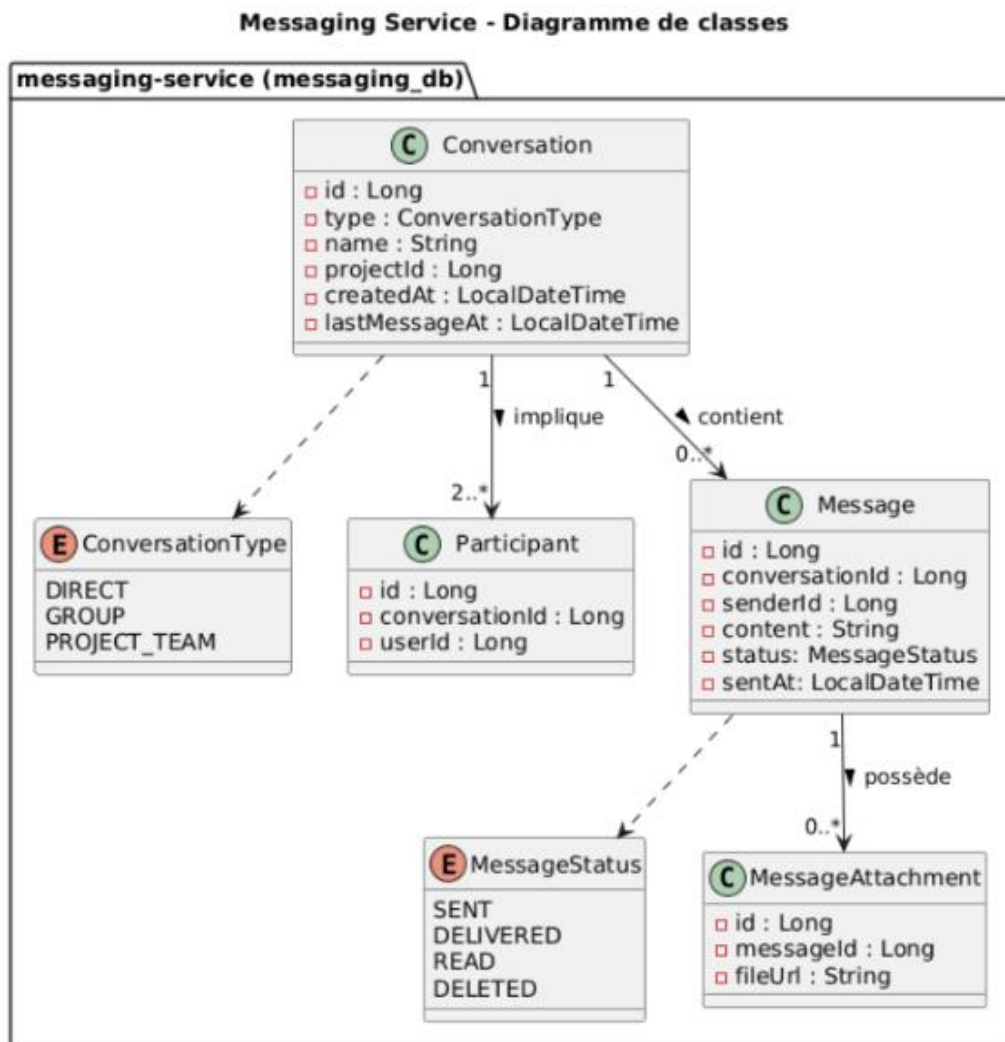
Le Messaging Service gère les conversations et les messages échangés entre utilisateurs. Il supporte des conversations directes, de groupe ou liées à une équipe projet.

Classes principales

- Conversation : représente une discussion (type, nom, projet associé).
- Participant : représente un utilisateur participant à une conversation.
- Message : message envoyé dans une conversation.
- MessageAttachment : pièce jointe associée à un message.
- MessageStatus (enum) : état du message (SENT, DELIVERED, READ, DELETED).
- ConversationType (enum) : type de conversation (DIRECT, GROUP, PROJECT_TEAM).

Relations entre classes

- Conversation (1) ---- (2..*) Participant : une conversation implique au minimum deux participants.
- Conversation (1) ---- (0..*) Message : une conversation contient plusieurs messages.
- Message (1) ---- (0..*) MessageAttachment : un message peut avoir plusieurs fichiers.
- Participant référence Conversation via conversationId.
- Message référence Conversation via conversationId.
- MessageAttachment référence Message via messageId.



6. Notification Service

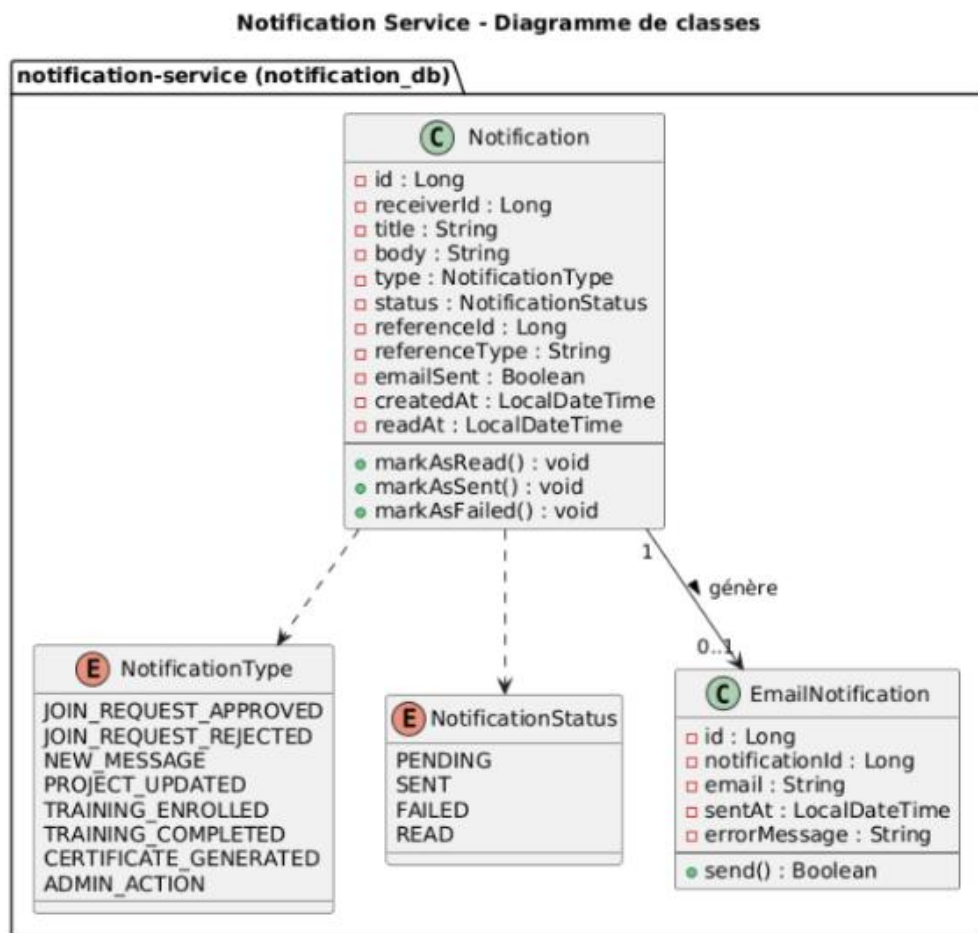
Le Notification Service est responsable de la création et de l'envoi des notifications. Il peut être appelé par d'autres services (Project, Messaging, Training, Admin). Il gère les notifications internes et l'envoi par email.

Classes principales

- Notification : notification envoyée à un utilisateur.
- EmailNotification : notification email liée à une notification interne.
- NotificationType (enum) : type d'événement (NEW_MESSAGE, JOIN_REQUEST_APPROVED, etc.).
- NotificationStatus (enum) : état d'envoi (PENDING, SENT, FAILED, READ).

Relations entre classes

- Notification (1) ---- (0..1) EmailNotification : une notification peut générer un email.
- EmailNotification référence Notification via notificationId.



7. Training Service

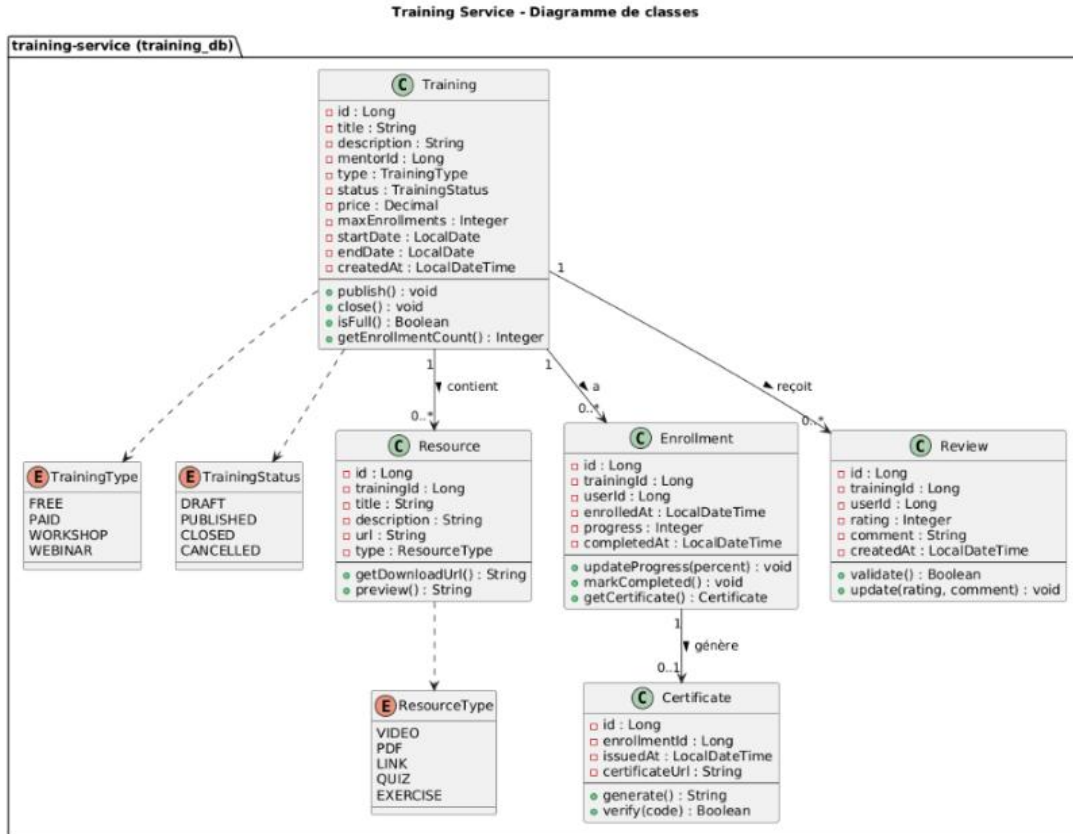
Le Training Service gère les formations proposées par les mentors : publication, ressources pédagogiques, inscriptions des étudiants, suivi de progression et génération de certificats. Les étudiants peuvent aussi laisser des avis notés.

Classes principales

- Training : formation (titre, description, mentorId, type, statut).
- Resource : ressource pédagogique liée à une formation.
- Enrollment : inscription d'un utilisateur à une formation.
- Certificate : certificat généré après complétion.
- Review : avis d'un étudiant sur une formation.
- TrainingType (enum) : type (FREE, PAID, WORKSHOP, WEBINAR).
- TrainingStatus (enum) : statut (DRAFT, PUBLISHED, CLOSED, CANCELLED).
- ResourceType (enum) : type de ressource (VIDEO, PDF, LINK, QUIZ, EXERCISE).

Relations entre classes

- Training (1) ---- (0..*) Resource : une formation contient plusieurs ressources.
- Training (1) ---- (0..*) Enrollment : une formation peut avoir plusieurs inscriptions.
- Training (1) ---- (0..*) Review : une formation reçoit plusieurs avis.
- Enrollment (1) ---- (0..1) Certificate : une inscription peut générer un certificat.
- Resource référence Training via trainingId.
- Enrollment référence Training via trainingId.
- Review référence Training via trainingId.
- Certificate référence Enrollment via enrollmentId.



8. Admin Service

Le Admin Service est responsable de la modération et du contrôle du système. Il permet de signaler des contenus (utilisateurs, projets, messages, formations), de sauvegarder les actions administratives (ban, suppression) et de conserver un audit complet.

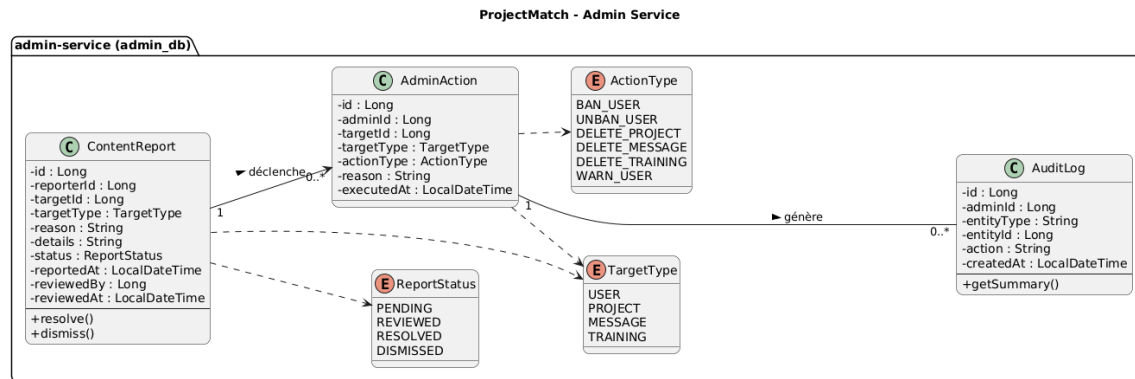
Classes principales

- ContentReport : signalement d'un contenu par un utilisateur.
- AdminAction : action effectuée par un administrateur.
- AuditLog : journal d'audit détaillé pour la traçabilité des actions administratives.
- ActionType (enum) : type d'action admin (BAN_USER, DELETE_PROJECT, etc.).
- ReportStatus (enum) : statut du report (PENDING, REVIEWED, RESOLVED, DISMISSED).
- TargetType (enum) : type de cible (USER, PROJECT, MESSAGE, TRAINING).

Relations entre classes

- ContentReport utilise TargetType et ReportStatus pour catégoriser le signalement.
- ContentReport → AdminAction : un signalement peut déclencher une ou plusieurs actions administratives (modération).

- AdminAction utilise ActionType et TargetType pour définir l'action appliquée.
- AdminAction ----> AuditLog : une action admin génère une entrée d'audit.
- AuditLog contient l'historique complet des actions administratives.



9. Conclusion

Cette architecture Microservices permet de séparer clairement les responsabilités, de faciliter l'évolutivité et de garantir une meilleure maintenabilité. Chaque service est autonome avec sa propre base de données, et la communication entre services se fait via API ou événements.